Reviewer Pack — Minimal Rank of K-theory and Communication Complexity Matrix…

Entry 407d1ee3ab14 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 17:49:06 UTC

Minimal Rank of K-theory and Communication Complexity Matrix Rank

Entry ID: `407d1ee3ab14` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 17:49:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every boolean function f with input size n, the rank of its associated vector bundle in algebraic K-theory is linearly correlated with its communication matrix rank, such that the rank of the vector bundle = Θ(communication_matrix_rank(f)).

Rationale (proposer's reasoning):

> Algebraic K-theory has been little applied to complexity theory, yet it provides a rich framework for studying vector bundles. Communication complexity is well-studied but lacks invariant measures beyond matrix rank. This conjecture aims to bridge these fields by exploring the potential of K-theoretic invariants for communication complexity.

Taxonomy category: `Algebraic K-Theory × Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9fdab589b2f6b9ad`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for a statistically significant number (N ≥ 100) of random boolean functions f with n ≤ 40, the ratio of vector bundle rank to communication matrix rank falls within [0.5, 2] and the correlation coefficient between the ranks is |ρ| ≥ 0.7. The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic K-theory AND communication complexity AND matrix rank - vector bundle in K-theory AND related to communication complexity matrix - 0-notation for rank in K-theory linked with communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.5s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Swap rows to avoid division by zero
            for j in range(i + 1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                raise ValueError("Matrix is singular")
        for j in range(n):
            if i != j:
                factor = Fraction(A[j][i], A[i][i])
                for k in range(n + 1):
                    A[j][k] -= factor * A[i][k]
    return A

def matrix_rank(A):
    n = len(A)
    rank = 0
    for i in range(n):
        if A[i][i] != 0:
            rank += 1
    return rank

def communication_matrix_rank(f, n):
    C = [[0] * (2**n) for _ in range(2**n)]
    for x in range(2**n):
        for y in range(2**n):
```

```
            if f(x) == f(y):
                C[x][y] = 1
    return matrix_rank(C)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_ranks = []
    communication_matrix_ranks = []

    for n in n_values:
        for _ in range(30):
            f = lambda x: random.choice([0, 1])
            vector_bundle_rank = sum(f(x) != f(y) for x in range(n) for y in range(n)) // (n * (n
            communication_matrix_rank_val = communication_matrix_rank(f, n)
            total_ranks.append(vector_bundle_rank)
            communication_matrix_ranks.append(communication_matrix_rank_val)

    mean_vector_bundle_rank = sum(total_ranks) / len(total_ranks)
    mean_communication_matrix_rank = sum(communication_matrix_ranks) / len(communication_matrix_ra
    correlation_coefficient = sum((x - mean_vector_bundle_rank) * (y - mean_communication_matrix_r

    conjecture_holds = 0.5 <= correlation_coefficient <= 2 and abs(correlation_coefficient) >= 0.7
    counterexample = "" if conjecture_holds else "correlation_out_of_bounds"

    return {
        "metric_name": "communication_matrix_rank",
        "metric_value": mean_communication_matrix_rank,
        "instances_tested": len(total_ranks),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(r
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_out_of_bounds\" first_failing_seed=
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means that it was not possible to verify the conjecture's conditions. | next: Run the test again with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13561
2	propose	ollama_remote	glm4:latest	0	0	12048
3	preregistration	ollama_remote	glm4:latest	0	0	10498
4	novelty	ollama_remote	glm4:latest	0	0	12232
5	novelty	ollama_remote	glm4:latest	0	0	9792
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21455
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11028
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15045
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17484
10	judge	ollama_remote	glm4:latest	0	0	13058

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 136199 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/407d1ee3ab14.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/407d1ee3ab14.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/407d1ee3ab14.tar.gz` (if generated)